

Установка Docker на Ubuntu 26.04 LTS

 serverspace.ru/articles/ustanovka-docker-na-ubuntu-26-04-lts

ООО «ИТГЛОБАЛКОМ ЛАБС»

May 24, 2026

Docker — один из самых распространённых инструментов для запуска контейнеризованных приложений. Он позволяет упаковать приложение вместе со всеми зависимостями в изолированную среду и запускать её одинаково на любой машине: локальной, тестовой или производственной. Именно поэтому Docker используют в разработке, CI/CD-пайплайнах и при развёртывании микросервисов.

Ubuntu 26.04 LTS вышла в апреле 2026 года и принесла обновлённый набор системных библиотек, ядро Linux 6.14 и длительную поддержку до 2031 года. В этой статье мы разберём, как установить Docker Engine и Docker Compose на свежую систему Ubuntu 26.04, настроить его для работы без привилегий суперпользователя и проверить корректность установки.

Основные понятия

Перед тем как переходить к командам, стоит определиться с терминами — особенно если вы работаете с Docker впервые.

Контейнер — изолированный процесс, запущенный в пространстве пользователя. Контейнер использует ядро хостовой операционной системы, но при этом имеет собственную файловую систему, сетевой стек и переменные окружения. В отличие от виртуальной машины, контейнер не эмулирует оборудование — он работает напрямую с ядром, поэтому запускается быстро и потребляет меньше ресурсов.

Образ (image) — шаблон, на основе которого создаётся контейнер. Образ включает операционную систему, приложение и его зависимости, собранные в слоистую структуру. Образы неизменяемы: каждый раз при запуске из одного образа получается одинаковый контейнер.

Docker Hub — публичный реестр образов. Здесь хранятся официальные образы Nginx, PostgreSQL, Node.js и тысячи других. Вы можете использовать готовые образы или публиковать собственные.

Docker Engine — основной компонент платформы. Он состоит из демона dockerd, REST API и клиента командной строки docker. Именно Engine отвечает за создание, запуск и остановку контейнеров.

Docker Compose — инструмент для описания и запуска многоконтейнерных приложений через YAML-файл. Вместо того чтобы запускать каждый контейнер

вручную отдельной командой, вы описываете всю инфраструктуру в одном файле и поднимаете её одной командой.

containerd — низкоуровневая среда выполнения контейнеров, на которую опирается Docker Engine. Начиная с Docker 20+, она устанавливается как отдельный пакет и управляется независимо.

Как работает Docker

Docker использует несколько механизмов ядра Linux, чтобы изолировать процессы друг от друга и от хостовой системы.

Namespaces ограничивают область видимости процессов: каждый контейнер видит только свою файловую систему, свои процессы и свой сетевой интерфейс. Механизм cgroups (control groups) ограничивает потребление ресурсов — CPU, памяти и дискового ввода-вывода. UnionFS отвечает за слоистую файловую систему образов: каждый слой хранится отдельно и повторно используется разными образами, что экономит место на диске.

Когда вы выполняете команду `docker run`, клиент отправляет запрос демону `dockerd` через Unix-сокеты. Демон проверяет, есть ли нужный образ локально. Если нет — скачивает его с реестра. После этого `containerd` создаёт контейнер, настраивает сетевые интерфейсы и запускает процесс внутри изолированного окружения.

На практике это означает, что приложение, запущенное в контейнере, не знает о соседних контейнерах и не может случайно повлиять на них. При этом взаимодействие между контейнерами возможно — через виртуальные сети Docker или явно пробрасываемые порты.

Требования к системе

Для установки Docker на Ubuntu 26.04 LTS понадобится:

- 64-битная архитектура: amd64, arm64 или armhf
- Минимум 2 ГБ оперативной памяти (рекомендуется 4 ГБ и более)
- Доступ к интернету для загрузки пакетов и образов
- Пользователь с правами `sudo`

Проверить архитектуру системы можно командой:

```
uname -m
```

Если вывод — `x86_64`, система подходит. Убедитесь также, что ядро актуально:

```
uname -r
```

Ubuntu 26.04 поставляется с ядром 6.14, которое полностью совместимо с актуальными версиями Docker.

Пошаговая установка Docker

Шаг 1. Удаление устаревших версий

Если на машине уже установлены старые версии Docker или совместимые пакеты из официального репозитория Ubuntu, их нужно удалить. Пакеты с именами `docker.io`, `docker-compose` или `containerd` конфликтуют с официальными пакетами Docker Inc.

```
for pkg in docker.io docker-doc docker-compose docker-compose-v2 podman-docker
containerd runc; do
    sudo apt-get remove -y $pkg
done
```

Конфигурационные файлы и данные при этом не удаляются. Если вам нужна чистая установка без сохранения данных предыдущих контейнеров, дополнительно выполните:

```
sudo rm -rf /var/lib/docker
sudo rm -rf /var/lib/containerd
```

На свежей системе этот шаг можно пропустить.

Шаг 2. Обновление пакетной базы и установка зависимостей

Сначала обновляем список пакетов:

```
sudo apt-get update
```

Затем устанавливаем пакеты, необходимые для работы с HTTPS-репозиториями и GPG-ключами:

```
sudo apt-get install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

Эти пакеты, как правило, уже присутствуют в Ubuntu 26.04, но явная установка гарантирует их наличие.

Шаг 3. Добавление GPG-ключа Docker

Docker распространяет подписанные пакеты. Чтобы APT мог проверить их подлинность, нужно добавить GPG-ключ репозитория:

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Ключ сохраняется в директорию `/etc/apt/keyrings/` — стандартное место для ключей сторонних репозиторийев начиная с Ubuntu 22.04.

Шаг 4. Подключение официального репозитория Docker

Добавляем репозиторий Docker в список источников APT:

```
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Команда автоматически определяет кодовое имя дистрибутива через переменную `VERSION_CODENAME`. Для Ubuntu 26.04 это будет `quokka`. Если репозиторий Docker ещё не добавил явную поддержку этой версии, APT может откатиться к ближайшей поддерживаемой — в этом случае всё равно будут установлены актуальные пакеты.

Обновляем индекс пакетов, чтобы APT увидел новый репозиторий:

```
sudo apt-get update
```

Шаг 5. Установка Docker Engine

Устанавливаем Docker Engine вместе с сопутствующими компонентами:

```
sudo apt-get install -y \  
docker-ce \  
docker-ce-cli \  
containerd.io \  
docker-buildx-plugin \  
docker-compose-plugin
```

Что входит в этот набор:

- `docker-ce` — сам Docker Engine (Community Edition)
- `docker-ce-cli` — клиент командной строки
- `containerd.io` — среда выполнения контейнеров
- `docker-buildx-plugin` — расширенный инструмент сборки образов с поддержкой мультиплатформенности
- `docker-compose-plugin` — Docker Compose v2, интегрированный как плагин CLI

Шаг 6. Запуск и автозапуск службы Docker

После установки демон Docker нужно запустить и добавить в автозагрузку, чтобы он стартовал при каждой перезагрузке системы:

```
sudo systemctl enable docker
sudo systemctl start docker
```

Проверяем статус:

```
sudo systemctl status docker
```

Ожидаемый вывод содержит строку `Active: active (running)`. Если служба не запускается, стоит проверить журнал:

```
sudo journalctl -u docker --no-pager -n 50
```

Шаг 7. Проверка установки

Запускаем тестовый контейнер `hello-world` — это самый быстрый способ убедиться, что Docker работает корректно:

```
sudo docker run hello-world
```

Docker скачает минимальный образ с Docker Hub и запустит контейнер, который выведет сообщение:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image...
4. The Docker daemon streamed that output to the Docker client...

Установка прошла успешно. Теперь можно настроить доступ без `sudo`.

Шаг 8. Работа с Docker без sudo

По умолчанию взаимодействие с демоном Docker требует прав суперпользователя, потому что Unix-сокеты `/var/run/docker.sock` принадлежат группе `docker`. Добавьте своего пользователя в эту группу:

```
sudo usermod -aG docker $USER
```

Чтобы изменение вступило в силу, нужно перезайти в систему или применить группу к текущей сессии:

```
newgrp docker
```

Проверяем, что Docker работает без sudo:

```
docker run hello-world
```

Важно понимать: пользователи группы docker фактически получают привилегии, сопоставимые с правами root. Добавляйте в группу только доверенных пользователей. На production-серверах этот шаг стоит взвесить отдельно.

Шаг 9. Проверка Docker Compose

Docker Compose v2 устанавливается как плагин к CLI и вызывается через docker compose (без дефиса):

```
docker compose version
```

Пример вывода:

```
Docker Compose version v2.27.0
```

Версия может отличаться — это нормально. Если нужна совместимость со старыми скриптами, которые вызывают docker-compose через дефис, можно создать символическую ссылку:

```
sudo ln -s /usr/libexec/docker/cli-plugins/docker-compose /usr/local/bin/docker-compose
```

Практический пример: запуск Nginx в контейнере

Чтобы убедиться, что всё работает как надо, запустим простой веб-сервер Nginx. Это хороший способ проверить не только установку Docker, но и проброс портов между контейнером и хостом.

```
docker run -d -p 8080:80 --name test-nginx nginx:alpine
```

Разберём флаги:

- -d — запуск в фоновом режиме (detached)
- -p 8080:80 — порт 8080 хоста проксируется на порт 80 контейнера
- --name test-nginx — задаём имя контейнеру для удобства управления
- nginx:alpine — образ на базе Alpine Linux, минимальный по размеру

Проверяем, что контейнер запущен:

```
docker ps
```

Открываем браузер или выполняем запрос:

```
curl http://localhost:8080
```

В ответ придёт стандартная страница приветствия Nginx. Останавливаем и удаляем тестовый контейнер:

```
docker stop test-nginx
docker rm test-nginx
```

Полезные команды для ежедневной работы

После установки Docker полезно сразу познакомиться с несколькими базовыми командами — они нужны постоянно.

Команда	Что делает
docker ps	Показывает запущенные контейнеры
docker ps -a	Все контейнеры, включая остановленные
docker images	Список загруженных образов
docker pull <image>	Скачать образ с Docker Hub
docker logs <container>	Посмотреть логи контейнера
docker exec -it <container> bash	Войти в работающий контейнер
docker stop <container>	Остановить контейнер
docker rm <container>	Удалить остановленный контейнер
docker rmi <image>	Удалить образ
docker system prune	Очистить неиспользуемые ресурсы

Возможные проблемы при установке

Репозиторий не найден или недоступен

Если команда `apt-get update` выдаёт ошибку после добавления репозитория Docker, проверьте кодовое имя дистрибутива вручную:

```
. /etc/os-release && echo $VERSION_CODENAME
```

Если Docker ещё не добавил поддержку Ubuntu 26.04 в свой репозиторий, замените `$VERSION_CODENAME` на `noble` (Ubuntu 24.04) вручную в файле `/etc/apt/sources.list.d/docker.list`. Пакеты при этом будут работать корректно — Docker Engine не привязан к конкретным версиям Ubuntu в части зависимостей.

Служба не запускается после установки

Такое иногда происходит при конфликте с уже запущенным containerd или другой реализацией среды выполнения. Проверяем статус:

```
sudo systemctl status containerd
sudo systemctl restart docker
```

Если ошибка повторяется, смотрим полный журнал:

```
sudo journalctl -u docker -f
```

Permission denied при работе без sudo

Если после добавления пользователя в группу docker команды по-прежнему требуют sudo, скорее всего, сессия не обновилась. Выполните полный выход и вход заново, или примените группу командой `newgrp docker`. Проверить текущие группы пользователя можно командой `groups`.

Что дальше

После успешной установки Docker Engine открываются несколько очевидных направлений для дальнейшей работы. Если планируете разворачивать многосервисные приложения — начните с Docker Compose: опишите сервисы в файле `docker-compose.yml` и управляйте ими как единым целым. Для работы с собственными образами нужно познакомиться с Dockerfile — файлом, в котором описывается сборка образа шаг за шагом.

Если задача — управление кластером контейнеров на нескольких машинах, следующий шаг — Docker Swarm или Kubernetes. Оба инструмента используют Docker-образы, так что базовые знания работы с Docker здесь будут только полезны.

На сервере в production-окружении стоит также настроить ротацию логов контейнеров. По умолчанию Docker пишет логи в JSON-файлы без ограничения размера, что может со временем переполнить диск. Добавьте в `/etc/docker/daemon.json` следующую конфигурацию:

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "50m",
    "max-file": "3"
  }
}
```

После изменения файла перезапустите демон:

```
sudo systemctl restart docker
```

Итог

Мы установили Docker Engine и Docker Compose на Ubuntu 26.04 LTS из официального репозитория Docker, запустили и проверили работу демона, настроили доступ без sudo и протестировали запуск контейнеров. Весь процесс занимает около десяти минут на чистой системе.

Docker через официальный репозиторий предпочтительнее пакета docker.io из Ubuntu потому, что он всегда актуален: обновления выходят быстрее, а версия движка не привязана к циклу выпусков дистрибутива. Для production-систем это важно.

Оценка: 4 из 5

Average rating : 4.8

Оценок: 6